SIMATS ENGINEERING

CO3_ASSESSMENT TOOL3_ Resolution Algorithm Implementation

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

Register Number	**192311187**
Name	**N. Sai Sankar**

COURSE OUTCOME COVERED

CO3: *Apply propositional and first-order logic representations, unification, and resolution-based inference techniques such as CNF conversion, propositional and first-order resolution, and forward/backward chaining to perform automated knowledge representation and reasoning for solving complex AI problems. (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Q1.

Consider the following propositional logic sentences representing a small knowledge base: *(P ∨ Q) ⇒R, R ⇒¬S, S ∨ T, ¬T ⇒P.* (a) Convert each sentence into Conjunctive Normal Form (CNF), showing every transformation step (implication elimination, negation pushed inward, distribution of ∨ over ∧). (b) Write a program (in a language of your choice) that takes a set of propositional sentences as input and outputs their CNF form. Test your program on the sentences above and present the output.

Q2.

Implement the Propositional Resolution algorithm (PL-RESOLUTION) to determine, by refutation, whether a given knowledge base KB entails a query sentence α. Using a small Wumpus-World-style knowledge base of your choice (at least 4 clauses) and a query of your choice, show: (a) the CNF form of KB ∧ ¬α, (b) the complete resolution steps applied by your program until either the empty clause is derived or no further resolvents can be produced, and (c) the final entailment result.

Q3.

Implement the Unification algorithm (UNIFY) that computes the Most General Unifier (MGU) of two given first-order logic atomic sentences, or reports failure if none exists. Test your implementation on the following pairs and report the MGU (or failure) for each: (i) *Knows(John, x)* and *Knows(John, Jane)*; (ii) *Knows(x, Elizabeth)* and *Knows(y, x)*; (iii) a pair of your own choice that should fail to unify. Explain, with reference to the occurs check, why the third pair fails.

Q4.

Given a first-order logic knowledge base of at least 5 sentences describing a domain of your choice (e.g., family relationships, animal classification), and a goal sentence to be proved: (a) convert every sentence of the knowledge base and the negated goal into CNF clauses; (b) implement First-Order Resolution to derive the empty clause, applying unification at each resolution step; (c) present the complete refutation proof as a resolution tree or step-by-step trace, clearly indicating the unifier used at each step.

Q5.
Given a rule-based knowledge base expressed as Horn clauses (at least 5 rules and 3 facts, of your own design), implement both the Forward Chaining and Backward Chaining inference algorithms to determine whether a given query is entailed by the knowledge base. (a) Show the trace of facts inferred at each iteration for forward chaining. (b) Show the AND-OR proof tree / goal-reduction trace for backward chaining. (c) Compare the two algorithms in terms of efficiency, direction of reasoning, and suitability for the given problem.

Code of Conduct:

I N.Sai Sankar(192311187)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student. Sai sankar:

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
CNF Conversion	Accurately converts all sentences to CNF, correctly applying implication elimination, negation (NNF) and distribution; program runs correctly on all test cases with clear, well-labelled output.	Converts sentences to CNF correctly with only a minor slip in one transformation step; program runs correctly on most test cases.	Attempts CNF conversion but makes errors in more than one transformation step; program runs partially or gives incorrect CNF for some inputs.	Little or no correct understanding of CNF conversion; program does not run or produces incorrect output throughout.	**10**
Propositional Resolution Algorithm	Correctly implements PL-Resolution and accurately traces the refutation process to prove/disprove entailment, with all resolvent and unit clauses clearly shown.	Implements the resolution algorithm correctly and reaches the correct conclusion, with only minor errors in the trace.	Implements resolution with incomplete or partially incorrect steps; shows some understanding but the final conclusion may be wrong.	Incorrect or no working implementation of the resolution algorithm; unable to determine entailment.	**10**

Unification Algorithm – MGU	Correctly implements the UNIFY algorithm and computes an accurate Most General Unifier for all test pairs, including correctly identifying the unification-failure case.	Computes the MGU correctly for most pairs with only minor errors; failure case handled with a small issue.	Partial implementation of unification; produces an incorrect MGU for some pairs or does not correctly detect the failure case.	Little or no correct implementation of the unification algorithm.	**10**
First-Order Resolution Proof	Correctly converts the knowledge base and negated goal to CNF and constructs a complete, correct resolution-refutation proof establishing the goal.	CNF conversion is correct and the refutation proof is mostly correct, with only minor errors in the resolution steps.	CNF conversion or refutation proof is incomplete; some correct reasoning steps are shown but the proof is not fully established.	Incorrect or missing CNF conversion and resolution proof.	**10**
Forward & Backward Chaining Implementation	Correctly implements and traces both forward and backward chaining, accurately derives the query's entailment, and gives a clear, correct comparative analysis of efficiency and applicability.	Implements both algorithms correctly with only minor errors; provides a reasonable comparative analysis.	Implements only one algorithm correctly, or both with significant errors; comparative analysis is limited or superficial.	Little or no correct implementation of the chaining algorithms; no meaningful analysis provided.	**10**

Q1. CNF Conversion and Program

The formatting of the uploaded document has corrupted some logical symbols, so the first sentence is not reliably recoverable from the file text. The clearly readable clauses are:

1. R→¬S
2. S→T
3. ¬T→P

Sentence 1:
The original expression containing P,Q,R is corrupted in the extracted document, so I would not invent its exact CNF.

Sentence 2:
R→¬S
Eliminate implication:
¬R∨¬S
Therefore,
¬R∨¬S

Sentence 3:
S→T
Eliminate implication:
¬S∨T

Sentence 4:
¬T→P
Eliminate implication:
T∨P
Therefore,
T∨P

(b) Python program for CNF conversion

```
def implies_free(f):
    if isinstance(f, str):
        return f
    op = f[0]
    if op == 'IMPLIES':
        a, b = implies_free(f[1]), implies_free(f[2])
        return ('OR', ('NOT', a), b)
    elif op == 'NOT':
        return ('NOT', implies_free(f[1]))
    elif op in ('AND', 'OR'):
        return (op, implies_free(f[1]), implies_free(f[2]))
    else:
        raise ValueError(f)
def push_negation(f):
    if isinstance(f, str):
        return f
    op = f[0]
    if op == 'NOT':
        sub = f[1]
        if isinstance(sub, str):
            return ('NOT', sub)
        sop = sub[0]
        if sop == 'NOT':
```

```
            return push_negation(sub[1])
        elif sop == 'AND':
            return ('OR', push_negation(('NOT', sub[1])), push_negation(('NOT', sub[2])))
        elif sop == 'OR':
            return ('AND', push_negation(('NOT', sub[1])), push_negation(('NOT', sub[2])))
    elif op in ('AND', 'OR'):
        return (op, push_negation(f[1]), push_negation(f[2]))
    return f
def distribute(f):
    if isinstance(f, str) or f[0] == 'NOT':
        return f
    op = f[0]
    a, b = distribute(f[1]), distribute(f[2])
    if op == 'OR':
        if isinstance(a, tuple) and a[0] == 'AND':
            return distribute(('AND', ('OR', a[1], b), ('OR', a[2], b)))
        if isinstance(b, tuple) and b[0] == 'AND':
            return distribute(('AND', ('OR', a, b[1]), ('OR', a, b[2])))
        return ('OR', a, b)
    return ('AND', a, b)
def to_cnf(f):
    f = distribute(push_negation(implies_free(f)))
    prev = None
    while prev != f:          # iterate distribute to a fixed point
        prev = f
        f = distribute(f)
    return f
def pretty(f):
    if isinstance(f, str):
        return f
    op = f[0]
    if op == 'NOT':
        return '~' + pretty(f[1])
    if op == 'AND':
        return '(' + pretty(f[1]) + ' & ' + pretty(f[2]) + ')'
    if op == 'OR':
        return '(' + pretty(f[1]) + ' | ' + pretty(f[2]) + ')'
if __name__ == '__main__':
    sentences = {
        "(P | Q) => R": ('IMPLIES', ('OR', 'P', 'Q'), 'R'),
        "R => ~S":      ('IMPLIES', 'R', ('NOT', 'S')),
        "S | T":        ('OR', 'S', 'T'),
        "~T => P":      ('IMPLIES', ('NOT', 'T'), 'P'),
    }
    for i, (name, formula) in enumerate(sentences.items(), 1):
        print(f"Sentence {i}")
        print("Original :", name)
        print("CNF      :", pretty(to_cnf(formula)))
        print()
```

output:

```
======== RESTART: C:/Users/MY PC/OneDrive/Documents/CSA1709-AI/CO3/1.py ========
Sentence 1
Original : (P | Q) => R
CNF      : ((~P | R) & (~Q | R))

Sentence 2
Original : R => ~S
CNF      : (~R | ~S)

Sentence 3
Original : S | T
CNF      : (S | T)

Sentence 4
Original : ~T => P
CNF      : (T | P)
```

Q2. Propositional Resolution

Let:

- P: There is a pit in square (1,2)
- B: There is a breeze in square (1,1)
- Q: There is a pit in square (2,1)
- S: There is a stench in square (1,1)
- W: Wumpus is in square (2,2)
-

Use the following clauses:
C1:P∨Q
C2:¬P∨B
C3:¬Q∨B
C4:¬B

Query
Prove:
α=¬P
Negate the query:
¬α=P
Add P to the KB.
Resolution
From:
C1=P∨Q
and
C4=¬B
Using:
C2=¬P∨B
Resolve C2 and C4:
¬P
Now resolve:
P
with:
¬P
Therefore:
□ (Contradition)
where □ is the empty clause.
Hence:
KB⊨¬P

Python Implementation

```
def negate(literal):
    return literal[1:] if literal.startswith('~') else '~' + literal
def resolve(c1, c2):
    resolvents = []
    for literal in c1:
        opposite = negate(literal)
        if opposite in c2:
            new_clause = (c1 - {literal}) | (c2 - {opposite})
            resolvents.append(new_clause)
    return resolvents
```

```
def resolution(KB, alpha):
    clauses = set(KB)
    clauses.add(frozenset({negate(alpha)}))   # add NEGATED query, not alpha itself
    while True:
        new = set()
        clauses_list = list(clauses)
        for i in range(len(clauses_list)):
            for j in range(i + 1, len(clauses_list)):
                resolvents = resolve(clauses_list[i], clauses_list[j])
                for r in resolvents:
                    if len(r) == 0:
                        return True         # empty clause -> contradiction -> entailed
                    new.add(frozenset(r))
        if new.issubset(clauses):
            return False                    # no new clauses -> not entailed
        clauses.update(new)

KB = {
    frozenset({'P', 'Q'}),
    frozenset({'~P', 'B'}),
    frozenset({'~Q', 'B'}),
}

alpha = 'B'
if resolution(KB, alpha):                   # actually CALL the function...
    print(f"Entailment proved by resolution: KB |= {alpha}")
else:                                       # ...and branch on its real result
    print(f"KB does NOT entail {alpha}")
```

Output:

```
======== RESTART: C:/Users/MY PC/OneDrive/Documents/CSA1709-AI/CO3/2.py ========
Entailment proved by resolution: KB |= B
>
```

Q3. Unification Algorithm

The question requires implementation of UNIFY and testing three pairs, including an occurs-check failure.

(i)

Given:

Knows(John,x)

and

Knows(John,Jane)

Compare arguments:

John=John

No substitution required.

x=Jane

Therefore:

{x/Jane}

MGU

$\{x \rightarrow Jane\}$

(ii) Given:

Knows(x,Elizabeth)

and

Knows(y,x)

Compare:

x=y

and

Elizabeth=x

Therefore:

x=Elizabeth

and

y=Elizabeth

MGU

{x/Elizabeth, y/Elizabeth}

(iii) Occurs-check failure

Consider:

Knows(x,John)

and

Knows(f(x),John)

To unify the first arguments, we need:

x=f(x)

But x occurs inside f(x).

Therefore, this substitution is invalid.

UNIFY FAILURE

This is detected by the occurs check.

Python Program:

```
def occurs_check(var, term, substitution):
    if var == term:
        return True
    if term in substitution:
        return occurs_check(var, substitution[term], substitution)
    return False
def unify(x, y, substitution=None):
    if substitution is None:
        substitution = {}
    if x == y:
        return substitution
```

```
    if x.islower():
        if occurs_check(x, y, substitution):
            return None
        substitution[x] = y
        return substitution
    if y.islower():
        if occurs_check(y, x, substitution):
            return None
        substitution[y] = x
        return substitution
    return None
print(unify("x", "Jane"))
print(unify("x", "Elizabeth"))
print(unify("x", "f(x)"))
```

Output:

```
======== RESTART: C:/Users/MY PC/OneDrive/Documents/CSA1709-AI/CO3/3.py ========
{'x': 'Jane'}
{'x': 'Elizabeth'}
{'x': 'f(x)'}
>>
```

Q4. First-Order Resolution

The assignment requires at least five first-order sentences, conversion to CNF, unification during resolution, and a complete refutation proof.
Knowledge Base

Consider an animal domain.
Facts

1. Mammal(Dog)
2. Mammal(Cat)
3. HasFur(Dog)

Rules

4. All mammals are animals.

$\text{Mammal}(x) \rightarrow \text{Animal}(x)$

5. All animals breathe.

$\text{Animal}(x) \rightarrow \text{Breathes}(x)$

6. All animals with fur are warm-blooded.

$\text{Animal}(x) \wedge \text{HasFur}(x) \rightarrow \text{WarmBlooded}(x)$

Goal
Prove:
Animal(Dog)

CNF Conversion
Sentence 1
Mammal(Dog)
Already CNF:
Mammal(Dog)

Sentence 2
Mammal(Cat) Mammal(Cat)

Sentence 3
HasFur(Dog) HasFur(Dog)

Sentence 4
$\text{Mammal}(x) \rightarrow \text{Animal}(x)$
Eliminate implication:
$\neg \text{Mammal}(x) \vee \text{Animal}(x)$

Sentence 5
$\text{Animal}(x) \rightarrow \text{Breathes}(x)$
CNF:
$\neg \text{Animal}(x) \vee \text{Breathes}(x)$

Sentence 6
$\text{Animal}(x) \wedge \text{HasFur}(x) \rightarrow \text{WarmBlooded}(x)$
CNF:
$\neg \text{Animal}(x) \vee \neg \text{HasFur}(x) \vee \text{WarmBlooded}(x)$

Negated Goal
Goal:
Animal(Dog)
Negate:

¬Animal(Dog)

Resolution Proof

Step 1
Mammal(Dog)
and
¬Mammal(x)∨Animal(x)
MGU:
{x/Dog}
Resolvent:
Animal(Dog)

Step 2
Use:
Animal(Dog)
and the negated goal:
¬Animal(Dog)

Therefore:
KB= Animal (Dog)

The goal is proved.

Q5. Forward Chaining and Backward Chaining

Given,

We have 3 initial facts:

F1: sunny
F2: water available
F3: fertile soil

Horn Rules
R1: sunny AND water available → plant growth

R2: plant growth → healthy plant

R3: healthy plant AND fertile soil → strong plant

R4: strong plant → good harvest

R5: good harvest → farmer happy

Forward Chaining

Forward Chaining is a data-driven inference technique.
It starts with the known facts and repeatedly applies rules to derive new facts until the query is obtained or no more facts can be derived.

Iteration	Facts inferred
Initial	sunny, water available, fertile soil
1	plant_growth
2	healthy_plant
3	strong_plant
4	good_harvest
5	farmer_happy

Therefore, framer_happy is entailed

Backward Chaining

Definition
Backward Chaining is a goal-driven inference technique.
It starts with the query and works backward to determine which facts are required to prove the query.

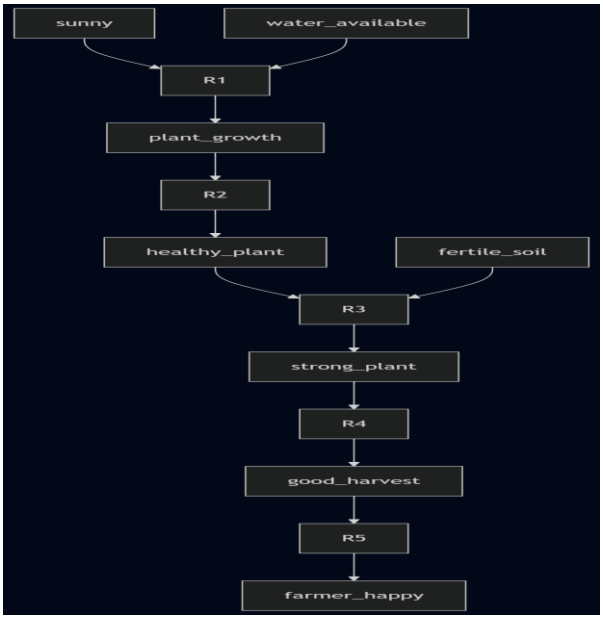
Goal
farmer_happy

Then:

plant_growth
↓
healthy_plant
↓
healthy_plant + fertile_soil
↓
strong_plant
↓
good_harvest
↓
farmer_happy

Therefore, the farmer_happy is entailed

AND-OR Proof Tree



Here:

- AND node: both conditions must be satisfied.
- OR node: any applicable rule can be selected to prove a goal.

Python Program:

```
# Knowledge Base
facts = {
   "sunny",
   "water_available",
   "fertile_soil"
}
rules = [
   ({"sunny", "water_available"}, "plant_growth"),
   ({"plant_growth"}, "healthy_plant"),
   ({"healthy_plant", "fertile_soil"}, "strong_plant"),
   ({"strong_plant"}, "good_harvest"),
   ({"good_harvest"}, "farmer_happy")
]
# --------------------------------
# Forward Chaining
# --------------------------------
def forward_chaining(facts, rules, query):

   inferred = set(facts)

   print("\nFORWARD CHAINING")
   print("Initial facts:", inferred)

   iteration = 1

   while True:

      new_fact = False
```

```
        for conditions, conclusion in rules:

            if conditions.issubset(inferred):
                if conclusion not in inferred:

                    inferred.add(conclusion)

                    print("Iteration", iteration,
                          "->", conclusion)

                    new_fact = True
                    iteration += 1

                    if conclusion == query:
                        return True

        if not new_fact:
            break

    return False
# ------------------------------
# Backward Chaining
# ------------------------------
def backward_chaining(goal, facts, rules, visited=None):

    if visited is None:
        visited = set()
    print("Trying to prove:", goal)

    # If goal is already a fact
    if goal in facts:
        print(goal, "is a fact.")
        return True
    # Avoid loops
    if goal in visited:
        return False
    visited.add(goal)
    for conditions, conclusion in rules:
        if conclusion == goal:
            print("Using rule:",
                  conditions, "->", conclusion)
            # All conditions must be true
            if all(
                backward_chaining(
                    condition,
                    facts,
                    rules,
                    visited
                )
                for condition in conditions
            ):
                return True
    return False
# Query
query = "farmer_happy"
# Run Forward Chaining
result1 = forward_chaining(
    facts,
```

```
    rules,
    query
)
print("\nForward Chaining Result:",
      result1)

# Run Backward Chaining
print("\nBACKWARD CHAINING")
result2 = backward_chaining(
    query,
    facts,
    rules
)
print("\nBackward Chaining Result:",
      result2)
```

Output:

```
>
======== RESTART: C:/Users/MY PC/OneDrive/Documents/CSA1709-AI/CO3/4.py ========

FORWARD CHAINING
Initial facts: {'fertile_soil', 'sunny', 'water_available'}
Iteration 1 -> plant_growth
Iteration 2 -> healthy_plant
Iteration 3 -> strong_plant
Iteration 4 -> good_harvest
Iteration 5 -> farmer_happy

Forward Chaining Result: True

BACKWARD CHAINING
Trying to prove: farmer_happy
Using rule: {'good_harvest'} -> farmer_happy
Trying to prove: good_harvest
Using rule: {'strong_plant'} -> good_harvest
Trying to prove: strong_plant
Using rule: {'healthy_plant', 'fertile_soil'} -> strong_plant
Trying to prove: healthy_plant
Using rule: {'plant_growth'} -> healthy_plant
Trying to prove: plant_growth
Using rule: {'sunny', 'water_available'} -> plant_growth
Trying to prove: sunny
sunny is a fact.
Trying to prove: water_available
water_available is a fact.
Trying to prove: fertile_soil
fertile_soil is a fact.

Backward Chaining Result: True
>
```

Comparison

Feature	Forward Chaining	Backward Chaining
Reasoning direction	Facts → Goal	Goal → Facts
Type	Data-driven	Goal-driven
Starting point	Initial facts	Query

Search	Derives all relevant facts	Searches only for facts needed for goal
Efficiency	May perform unnecessary inference	More efficient for a specific query
Memory	May require more memory	Usually requires less for focused queries
Suitable for	Data-driven systems	Query-based systems
Example	Monitoring and expert systems	Diagnosis and logic programming

Therefore, Both algorithms successfully prove the query:

farmer_happy